

The Resource Divide: Data Scarcity, Compute Inequity, and the Future of Global LLM Governance

Connor Sempf

*School of Electrical and Computer Engineering
Georgia Institute of Technology
Atlanta, USA
csempf3@gatech.edu*

Bo Pang

*School of Electrical and Computer Engineering
Georgia Institute of Technology
Atlanta, USA
bpang42@gatech.edu*

Abstract—While large language models (LLMs) are positioned as universal tools, the resources required to train and deploy them are distributed highly unevenly across the world’s languages. This paper quantifies the compounding disadvantages faced by low-resource languages across three critical dimensions: tokenization efficiency, memory scalability, and hardware access. Using the Llama-3 tokenizer, we benchmark six languages and find extreme fertility disparities. Hindi requires 2.83 tokens per word and Swahili 2.34, versus 1.15 for English. This results in a 7.99× attention cost multiplier for Hindi compared to 1.32× for English. Through KV cache memory simulation across five hardware tiers from NVIDIA H100 to RTX 4060, we demonstrate that the RTX 4060 hits an out-of-memory wall at just 14,000 tokens while the H100 sustains 544,000 tokens, a 40× gap. In a realistic serving scenario with 32 concurrent users, Hindi conversations on an RTX 4060 OOM after a single turn, while English sustains 3 turns; on a T4, Hindi OOMs at turn 6 versus English at turn 13. Prefill-decode disaggregation analysis reveals that decode dominates 95-99.9% of end-to-end latency across all workload types, and heterogeneous clusters can halve serving costs by offloading decode to commodity GPUs. This shows a 12× latency penalty from on to the other. End-to-end inference reveals that processing Hindi on a T4 GPU is 26× slower than English on an H100 for an identical 100-word task. A comprehensive language ecosystem analysis combining speaker population, digital content volume, digital literacy, and LLM benchmark accuracy produces a Composite Resource Divide Index (CRDI) of 0.800 for Swahili versus 0.099 for English. We contextualize these findings against the reality that only 42 of approximately 7,000 world languages (0.6%) enjoy meaningful LLM support. The analysis offers a dual-viewpoint framework contrasting structural governance interventions with data-driven self-correction pathways, and delivers concrete policy recommendations spanning compute commons, targeted data investment, and inclusive tokenizer design.

I. INTRODUCTION

The rapid proliferation of large language models (LLMs) has centralized AI capabilities in a handful of nations and corporations despite open-weight releases [1]. The United States controls approximately 74% of global high-end AI compute, with China at 14% and the European Union at merely 5% [2]. An estimated 160 nations are classified as “compute deserts,” entirely dependent on foreign cloud infrastructure [3]. Within Africa, only 38% of the population used the internet in 2024, with rural penetration at just 23% versus 57% in urban areas [4]. Sub-Saharan Africa accounts for 52% of the global population facing digital exclusion [4]. This concentration is

not accidental: the U.S. CHIPS Act appropriates \$52.7 billion while explicitly prohibiting semiconductor expansion in China for ten years [5], and export controls on NVIDIA H100/H200 GPUs institutionalize access asymmetries.

A. AI Divide as a Global Risk

The AI divide extends beyond compute to data. English dominates 49.7% of global web content, while Swahili, which by the way is spoken by 200 million people, constitutes merely 0.0025% [6]. Hindi, with 600 million speakers, accounts for only 3.77% of internet content. This data imbalance creates a self-reinforcing cycle: less digital content leads to poorer language models, which reduces adoption, which discourages content creation [7].

B. Scope and Objectives

This paper investigates how compute and data disparities manifest as compounding, quantifiable performance gaps in LLM deployment. We quantify the “tokenization tax” (higher tokens-per-word ratios for low-resource languages), the “memory wall” (KV cache limits on consumer hardware), and their interaction in realistic serving scenarios. We further analyze the distinct hardware requirements of prefill (compute-bound) versus decode (memory-bound) phases, and evaluate heterogeneous cluster scheduling. Finally, we contextualize these technical findings within a comprehensive language ecosystem framework combining speaker population, digital content, literacy rates, and model performance.

C. Contributions

This paper makes three contributions. The first is about empirical quantification of compounding resource divide using real tokenizer benchmarks and memory simulation across hardware tiers. The second is on a serving-systems analysis of multi-turn conversational KV cache accumulation and prefill-decode disaggregation. The last is about a Composite Resource Divide Index (CRDI) synthesizing six metrics to rank global language AI readiness.

II. LITERATURE REVIEW

Recent scholarship shifts focus from data to the physical concentration of compute. GovAI [1] posits that high-end

compute is uniquely *excludable* and *detectable*, functioning as a primary lever for regulation. Lehdonvirta et al. [2] argue that nations without territorial clusters lack “computational sovereignty.” This concentration where 75% of capacity resides in the US [3][4], creates a “Compute Moat” that forces 160 nations into absolute API-dependency.

A. Geopolitics of Compute

GovAI identifies compute as uniquely excludable and detectable, making it the primary lever for AI regulation [1]. Lehdonvirta et al. argue that nations without territorial compute clusters lack “computational sovereignty” and remain perpetually dependent [2]. The Tony Blair Institute and UC Berkeley AI Policy Hub document how supply-chain concentration creates structural barriers that market mechanisms cannot self-correct [3][8].

B. Linguistic Inequity: The Tokenization Tax

Lundin et al. demonstrate that each additional token per word reduces downstream accuracy by 8–18 percentage points, coining the “tokenization tax” [7]. Ahuja et al. show that low-resource languages face systematic cost-performance disadvantages in LLM inference [9]. Our analysis extends this by showing that attention costs scale as $O(n^2)$ with token count, amplifying disparities quadratically.

C. Data Imbalance and Model Bias

Hooper et al. identify KV cache as the fundamental bottleneck for long-context inference [13]. vLLM’s PagedAttention reduces memory waste from 60–80% to under 4%, enabling 2–4× throughput improvements [14][15]. Recent work by WVA and DUET establishes that prefill and decode phases have fundamentally different hardware requirements motivating phase-specific hardware disaggregation [16][17]. HBM3e bandwidth at 3.35 TB/s versus GDDR6 at 272 GB/s creates a 12× gap that directly governs decode throughput [18].

D. The Memory Wall and Serving Systems

Joshi et al. and the Masakhane project document that of approximately 2,000 African languages, only about 42 have meaningful LLM support [10][11]. Swahili’s 200 million speakers have access to only 700 million digital tokens. This is clearly orders of magnitude less than English’s trillions. The “Stochastic Parrots” framework establishes that biases in training data encode systemic privileging of dominant languages and cultures [12]. African Languages Lab’s recent work on 40 languages with 19 billion tokens provides a foundation, yet remains minuscule compared to English corpora [11].

E. Language Resource Gaps

As training data reflects existing societal imbalance, and as a result, “people in position of privilege. . . tend to be overrepresented in training data” [12]. Additionally, “biases can be encoded in ways that form a continuum from subtle patterns” [12]. If we shift back to languages, the disparity is even more evident in low-resource language areas. Specifically, where 2144 of all 7111 living languages today are African. . . But

only a small portion of linguistic resources for NLP research are built for African languages. . . there are only few NLP publications” [10]. This lack of available data clearly limits the ability of ML models to generalize equally and effectively across all languages. This results in poor performance for those underrepresented languages and the people who speak them. Studies also show that LLMs trained on African languages consistently underperform compared to models trained on high-resource languages. The imbalance can also be measured in the performance of large language models. “Out of over 2,000 African languages, only about 42 have any support in existing LLMs. . . Major challenges include severe data scarcity” [11].

F. The Need for Data Equity

The UNESCO Recommendation on the Ethics of AI recognized this issue and states that “AI actors should play a participative and enabling role to ensure peaceful and just societies, which is based on an interconnected future for the benefit of all, consistent with human rights and fundamental freedoms” [20]. These studies show how the scarcity and imbalance of training data disproportionately disadvantage underrepresented groups. This manifests as performance gaps in AI systems where the underrepresented groups have access to poorer models. Focusing on the need for inclusive and equitable data practices will shrink the performance gap most effectively.

III. TECHNICAL EXPERIMENTS

A. Experimental Design

Our experiments employ the Llama-3 tokenizer (via HuggingFace Transformers) for real tokenization benchmarks, and a KV cache memory simulation framework modeling Llama-3-8B with 4-bit AWQ quantization across five hardware tiers: H100 (80GB HBM3, 3.35 TB/s), A100 (80GB HBM2e, 2.04 TB/s), RTX 3090 (24GB GDDR6X, 936 GB/s), T4 (16GB GDDR6, 320 GB/s), and RTX 4060 (8GB GDDR6, 272 GB/s). KV cache per token is 0.125 MB for FP16 precision (formula: $2 \times 32 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ head_dim} \times 2 \text{ bytes}$). Additionally, the last experiment runs its own training on the auto tokenizer and auto model for sequence classification from the Transformers library. This is done to test data constriction during training to see the change in outcomes of model accuracy.

B. Tokenization Fertility Results

Table I presents tokenization fertility for six languages. Hindi requires 2.46× more than English’s 1.15 at 2.83 tokens per word. Because attention scales quadratically with token count, the “attention cost multiplier” (*fertility*²) shows Hindi at a 6× penalty at 7.99× when compared to English at 1.32×. Swahili at 2.34 tokens/word incurs a 5.47× attention cost.

TABLE I
TOKENIZATION FERTILITY AND ATTENTION COST MULTIPLIER BY
LANGUAGE (LLAMA-3 TOKENIZER).

Language	Script	Tokens	Words	Fertility	Attn. Cost
Chinese	Han	76	104	0.73	0.53
English	Latin	61	53	1.15	1.32
Spanish	Latin	104	62	1.68	2.81
Swahili	Latin	131	56	2.34	5.47
Arabic	Arabic	108	45	2.40	5.76
Hindi	Devanagari	212	75	2.83	7.99

C. Memory Wall Simulation Results

KV cache grows linearly with sequence length: a 4K context at batch size 32 requires 15.4 GB (English) to 35.3 GB (Hindi). Table II shows maximum sustainable sequence lengths before OOM. The consumer GPU (RTX 4060) hits the memory wall

TABLE II
MAXIMUM SEQUENCE LENGTH BEFORE OOM (LLAMA-3-8B 4-BIT).

Hardware	VRAM	Max Seq (B=1)	Max Seq (B=32)
H100	80GB	544,768 tokens	17,024 tokens
A100	80GB	544,768 tokens	17,024 tokens
RTX 3090	24GB	131,891 tokens	4,121 tokens
T4	16GB	72,908 tokens	2,278 tokens
RTX 4060	8GB	13,926 tokens	435 tokens

at just 435 tokens for batch size 32, which comes out to less than one page of text. The H100 sustains 40× more concurrent tokens.

D. Combined End-to-End Latency Analysis

For an identical 100-word semantic task, end-to-end latency reveals compounding disadvantage. English on H100: 0.7s. Hindi on T4: 18.8s. This shows a 26× gap. The Resource Divide Index ($RDI = fertility^2 / (bandwidth \cdot VRAM)$) reaches 1,010× between extremes.

E. Multi-Turn Conversational Serving Simulation

Real-world LLM applications operate as multi-turn conversations where each turn appends to the KV cache. We simulate a serving scenario with 32 concurrent users, each engaging in a 10-turn conversation with 80 words per turn. Fig. 4(a) shows KV cache accumulation. Specifically, it shows that Hindi conversations generate 35.3 GB of KV cache by turn 10 versus 14.4 GB for English with both approaching the T4’s 8.9 GB limit. Fig. 4(b) presents the OOM heatmap. The RTX 4060 OOMs after just 1 turn for Hindi (batch=32) versus 3 turns for English. The T4 OOMs at turn 6 for Hindi versus turn 13 for English. Only frontier GPUs (H100, A100) sustain all 20 turns for all languages. This demonstrates that low-resource languages not only cost more per token but also reduce service capacity. As a result, a Hindi chatbot can serve 3× fewer concurrent users than an English equivalent before memory exhaustion.

F. Prefill-Decode Disaggregation Analysis

LLM inference comprises two fundamentally different phases [16][17]. Prefill processes all input tokens in parallel

via large matrix multiplications. This achieves 90–95% GPU utilization on H100. Decode generates one token at a time, reading the entire KV cache from memory each step while it is memory-bound with only 20–40% utilization. Table III breaks down latency by workload type. Decode dominates 95.6–99.9% of end-to-end latency across all workloads. For a medium essay (500 input, 300 output tokens), decode consumes 99.5% of time on all GPUs. Fig. 5 shows the Pareto

TABLE III
PREFILL VS DECODE LATENCY BREAKDOWN (LLAMA-3-8B 4-BIT).

Workload	GPU	TTFT (ms)	Decode (ms)	Decode %
Short Q	H100	1.8	52.9	96.7%
Short Q	T4	19.3	554.3	96.6%
Medium Essay	H100	2.4	534.7	99.5%
Medium Essay	A100	4.8	878.5	99.5%
Long Document	H100	10.3	959.7	98.9%
Code Gen	H100	2.0	1416.5	99.9%

frontier for disaggregated serving. Assigning prefill to H100 and decode to RTX 4060 halves serving cost (\$0.00185 vs 0.00373 per request) but increases latency by 12× (6,588 ms vs 537 ms). For cost-sensitive deployments in the Global South, this tradeoff is unavoidable. H100-class hardware is just unavailable.

G. Comprehensive Language Ecosystem Analysis

Fig. 6 presents a six-panel dashboard analyzing language AI readiness across ten major languages. We synthesize six metrics into a Composite Resource Divide Index (CRDI): speaker population, internet content percentage, digital literacy proxy, estimated LLM MMLU accuracy, tokenization fertility, and Wikipedia contributor count. The CRDI reveals striking disparities (Table IV). Swahili scores 0.800, despite 200 million speakers, due to minuscule digital content (0.0025%), low digital literacy proxy (38%), poor LLM accuracy (45%), and high fertility (2.34). Bengali follows at 0.779. English scores 0.099. That is an 8.1× advantage over Swahili. The correlation heatmap (Fig. 6e) reveals that internet content percentage correlates strongly with Wikipedia activity ($r=0.93$) and LLM accuracy ($r=0.66$), while fertility shows strong negative correlation with LLM accuracy ($r=-0.86$). This is evidence for confirming that tokenization cost directly predicts model performance.

TABLE IV
COMPOSITE RESOURCE DIVIDE INDEX (CRDI) AND DIGITAL
OPPORTUNITY INDEX (DOI).

Language	CRDI	DOI	Tier
Swahili	0.800	0.200	Low
Bengali	0.779	0.221	Low
Arabic	0.674	0.326	Moderate
Hindi	0.668	0.332	Moderate
Russian	0.653	0.347	Moderate
Spanish	0.607	0.393	High
French	0.588	0.412	High
Japanese	0.587	0.413	High
Chinese	0.386	0.614	High
English	0.099	0.900	High

H. Dataset Constriction and Model Accuracy

Fig. 7 Shows the results of training on the Sequence Classifier Model and its auto tokenizer from the Transformations library directly with the civil comments subset from the WILDS dataset. There are three sets of data constriction where the data set is limited to 10%, 20%, and 50% of its total capacity. Training is performed for each of these sets and final testing and model accuracy are output as graphs. This reveals how the data availability problem is definitely an issue for underrepresented datasets as the model accuracy increases with less constriction.

TABLE V
DATASET CONSTRICTION WITH MODEL ACCURACY AND F1 SCORES

Data Constriction	Accuracy	F1 Score
10%	0.9071	0.5911
20%	0.9145	0.5817
50%	0.9176	0.6023

IV. DUAL VIEWPOINT ANALYSIS

A. Viewpoint A (Bo Pang): Structural Governance Imperative

The experimental evidence supports an urgent need for structural intervention. The compute gap is not self-correcting: a T4 GPU’s GDDR6 memory cannot upgrade itself to HBM3e, and export controls prevent acquisition of frontier hardware [1][5]. The RTX 4060 OOMs after one turn of Hindi conversation with 32 concurrent users, not due to algorithmic inefficiency, but because 8 GB of GDDR6 is structurally insufficient for multilingual serving.

The tokenization tax is equally structural. Hindi’s 2.83 tokens per word stems from English-centric BPE training on 15 trillion tokens, a corpus where Hindi constitutes a negligible fraction [7]. Vocabulary extension experiments show Hindi reducible from 2.61 to 1.19 tokens per word with targeted tokenizer redesign [19], but market incentives do not fund such redesign because Hindi-speaking markets generate less revenue per user.

The CRDI data reinforces this argument. Swahili’s CRDI of 0.800 versus English’s 0.099 reflects a systemic failure across all six measured dimensions. Policy recommendations include: (1) Sovereign AI funds financing regional compute clusters; (2) International Compute Commons agreements modeled on CERN; (3) Mandatory inclusive tokenizer design for models above threshold parameter counts; (4) UNESCO-expanded governance monitoring CRDI-like indices.

B. Viewpoint B (Connor Sempf): Data-Driven Correction

Connor Sempf presents a contrasting perspective, arguing that performance gaps across regions/languages/cultures are primarily caused by data inequality. Viewpoint A does identify compute barriers, but it still risks overemphasizing hardware at the expense of data. This is the root cause of the poor LLM performance from this viewpoint. Disparities in performance of AI models and the ubiquitous access to better models are driven by the access to larger quantities of better data for

training. For example, the vast majority of available content for NLP training is in English, making English-speakers have access to better general purpose LLMs. On the other hand, exclusive Swahili or Farsi speakers will have comparatively less access to advanced models due to the lack of resources rooted in those languages that would enable proper training. Simply increasing the computing power alone will not fix this problem for areas that have less technological advancement when it comes to AI. Without intentional governance on dataspecific standards and the funding related to it, these disparities will persist or get worse with time.

The CRDI correlation analysis shows LLM accuracy strongly correlates with internet content ($r=0.66$) and negatively with fertility ($r=-0.86$). Without representative training, no compute quantity can bridge that gap. For example, a Swahili model trained on 700 million tokens will underperform regardless of whether inference runs on H100 or T4. The data argument is also empirically grounded. Masakhane and African Languages Lab demonstrate that targeted data collection (even modest-scale community efforts) improves performance substantially [10][11]. XBridge and other similar projects have shown that extension of vocabulary and multilingual pre-training reduce fertility premiums by 40–60% [19]. Market mechanisms can also prove to be an aid. For example, India’s 600 million Hindi speakers represent an addressable market. As the demand for Hindi LLM applications grows, commercial incentives will certainly drive further data collection.

The solution to this problem starts with targeted investment into data collection and access sources. Costs for curating data for less represented languages like Swahili is much much less than building larger clusters to fix the issue just with compute. Additionally, once the data is out there, it can’t be retracted unlike distributed computing resources.

V. CONCLUSION

A. Assessment

These explorations demonstrate that the AI divide is real, quantifiable, and compounding across tokenization, memory, hardware, and data ecosystems. Key findings include that Hindi pays a 7.99× attention cost penalty versus English’s 1.32×. Secondly, consumer GPUs OOM at 14K tokens while frontier GPUs sustain 544K, which is a 40× gap. Thirdly, in serving scenarios, Hindi on RTX 4060 OOMs after one conversation turn versus three for English. Additionally, decode dominates 95–99.9% of latency, making memory bandwidth the binding constraint. Lastly, the CRDI shows 8.1× structural disparity between Swahili (0.800) and English (0.099).

B. Future Actions

Moving forward requires coordinated action in three main areas. First, there needs to be an established international data commons for low-resource languages, modeled on Masakhane but scaled to billions of tokens. Second, an implementation of regional compute-sharing agreements and sovereign AI funds with CRDI-based metrics tracking progress needs to

be done. Third, a mandate for inclusive tokenizer design as standard practice (not afterthought) for models above threshold sizes must happen. The dual-viewpoint analysis shows neither markets nor governance alone suffices; only parallel investment in data equity and compute access can break the cyclic exclusion of the majority of the world’s languages from the LLM revolution.

REFERENCES

- [1] GovAI, “Computing Power and the Governance of Artificial Intelligence,” arXiv:2409.02888, 2024.
- [2] V. Lehdonvirta et al., “Compute North vs. Compute South: Computational Sovereignty and the AI Divide,” Proc. AIES, 2024.
- [3] Tony Blair Institute, “State of Compute Access: Measuring Global AI Infrastructure,” Policy Report, 2024.
- [4] Nature, “Bridging the Digital Divide: Digital Literacy and Access in Rural Communities in China and Nigeria,” 2026.
- [5] U.S. Congress, “CHIPS and Science Act of 2022,” Public Law 117-167, Aug. 2022.
- [6] W3Techs / Intelpoint, “English accounts for 49.40% of internet content,” 2024.
- [7] J. Lundin et al., “The Token Tax: Systematic Bias in Multilingual Tokenization,” arXiv:2509.05486, 2025.
- [8] UC Berkeley AI Policy Hub, “An Evolving AI Supply Chain,” Policy Report, 2024.
- [9] K. Ahuja et al., “Cost-Performance Optimization for Low-Resource Languages,” ACL, 2024.
- [10] M. Abbott et al., “Masakhane—Machine Translation for Africa,” arXiv:2003.11529, 2020.
- [11] A. Adebara et al., “The State of LLMs for African Languages,” arXiv:2506.02280, 2025.
- [12] E. Bender et al., “On the Dangers of Stochastic Parrots,” Proc. FAccT, 2021.
- [13] C. Hooper et al., “KVQuant: Towards 10 Million Context Length,” UC Berkeley EECS, 2024.
- [14] W. Kwon et al., “vLLM: Efficient Memory Management for LLM Serving with PagedAttention,” SOSP, 2023.
- [15] Introl, “KV Cache Optimization: Memory Efficiency for Production LLMs,” 2026.
- [16] WVA, “A Global Optimization Control Plane for LLM Inference,” arXiv:2603.09730, 2026.
- [17] DUET, “Disaggregated Hybrid Mamba-Transformer LLMs with Prefill and Decode-Specific Packages,” 2025.
- [18] Towards Data Science, “Prefill Is Compute-Bound, Decode Is Memory-Bound,” 2026.
- [19] A. Nanda et al., “Reducing Tokenization Premiums for Low-Resource Languages,” arXiv:2601.13328, 2026.
- [20] UNESCO, “Recommendation on the Ethics of Artificial Intelligence,” 2021.
- [21] European Parliament, “Regulation (EU) 2024/1689—AI Act,” 2024.

APPENDIX

A. Compute Usage

This project used both the Georgia Tech AI Makerspace and local development.

TABLE VI
RESOURCE UTILIZATION DURING DEVELOPMENT AND REPRODUCTION.

Resource	Usage
AI Makerspace Hours	~14 hours across 4 sessions
GPU Nodes	1
GPUs Used	RTX 3090 and “First Available” in PACE-ICE Setup
Total Compute Time	~20 hours (development + reproduction)

B. AI Usage

For section 1 (Introduction), AI was used as an assisting tool to find additional metrics that continued to support that background and motivation that was outlined in the initial progress report. This entailed asking for additional sources and possible quote searching for evidence, but it should still be noted that all final evidence, verification, and drawn conclusions were thought out, analyzed, drawn, and verified manually. It was used as a searching and structuring tool. General grammar, structuring, reference noting, and other formatting questions were asked during the formation of the introductory sections as well.

For section 2 (Literature Review), AI was used as an assisting tool to find additional sources beyond the provided list of sources in the project outline. Additionally, quote searching for evidence was used to help prove/disprove the two viewpoints in certain ways that could be noted in the report. General grammar, structuring, reference noting, and other formatting questions were asked during the formation of literature review sections. It should still be noted that all final evidence and conclusions used were manually verified.

For section 3 (Experiments), AI tools were used in two forms. The first was for sanity checks for general experimentation validity. This process is essentially asking questions about if certain assumptions about design constraints, experiment pipeline and architecture flow, general structure, and validity of results were in the right direction for the proving/disproving the viewpoints. For example, in Experiment 7, questions about tradeoffs and which certain pretrained model selections may be favored for the experiment, tradeoffs for loss function suitability, tradeoffs for epochs suitability, chosen constriction levels, etc. These questions were asked as forms of sanity checking the ideas and flow for the experiment. The other form was with code development. In general, this was saved for boilerplate and styling purposes like with the matplotlib library. Sometimes, unfamiliar syntactic areas garnered questions. For example, in Experiment 7, questions were related to unknown syntax details for the new unknown models used in the experiment with the auto tokenizer usage and model weight adjustments.

For section 4 (Dual-Viewpoint Analysis), half of the information was derived from the initial progress report where the setup for the viewpoints was initially laid out with the reference evidence like in the introductory section. Pretty much all of this was done before the final report so there was no assistance necessary for this part other than what was already described before for referencing assistance. The other half was about taking the new information from the experimental data and results to ground the viewpoints even further. AI was used minimally here since the data itself tells the story and could be just inserted into the sections. Assistance came in the form of asking questions about results confirmation and the tying-in nature back to the viewpoints. Again, all final evidence, verification, and drawn conclusions were thought out, analyzed, drawn, and verified manually. It

was used as a searching and structuring tool. General grammar, structuring, reference noting, and other formatting questions were asked during the formation of the these sections as well.

For section 5 (Conclusion), this was essentially the same as in Section 4's Dual-Viewpoint Analysis. The data made the conclusions apparent. AI was merely used for metric confirmation, grammar and structure, writing assistance, etc. Since it was never about confirming a winner, this section really just needed to reiterate the findings.

C. Data and Code Availability

All code, data, and figures are available in the project repository. These include the 7 experiment files, 18 image figures, and 7 json data files. There is also a README file for instructions on the repository, its contents, and the reproduceability of experimental results. It can be found at https://github.com/Pb314314/Fund_ML/tree/main